



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий

Кафедра КБ-4 «Интеллектуальные системы информационной безопасности»

Дисциплина «Технологии извлечения знаний из больших данных»

**Отчет
о проделанной практической работе**

Выполнил студент 1 курса
Группы: ББМО-01-25
*Мухаметшин Александр
Ринатович*

Москва
2025

ОГЛАВЛЕНИЕ

ТЕОРЕТИЧЕСКОЕ ВВЕДЕНИЕ	3
ЗАДАНИЕ	6
ХОД РАБОТЫ	7
ВЫВОД.....	26
ПРИЛОЖЕНИЕ А.....	27

ТЕОРЕТИЧЕСКОЕ ВВЕДЕНИЕ

Метод k ближайших соседей (k -Nearest Neighbors, KNN) - это простой непараметрический алгоритм машинного обучения с учителем, используемый для задач классификации и регрессии. Он основан на гипотезе компактности: похожие объекты находятся близко друг к другу в пространстве признаков. KNN относится к ленивым алгоритмам (lazy learning), поскольку не строит явную модель во время обучения, а хранит весь обучающий набор данных и вычисляет предсказания на этапе тестирования.

При разработке KNN-классификатора используется учебный набор данных $U = \{(z_1, y_1), \dots, (z_s, y_s)\}$, в котором каждый кортеж (z_i, y_i) содержит информацию об объекте $z_i \in Z$ и метку класса $y_i \in Y = \{1, 2, \dots, R\}$, определяющую класс, к которому принадлежит объект z_i . Набор объектов Z представляет собой объединение наборов объектов разных классов Z^r ($r = 1, \dots, R$). Каждый объект $z_i \in Z$ представлен q -мерным вектором числовых характеристик $z_i = (z_{i1}, \dots, z_{iq})$, (нормированных значениями из отрезка $[0, 1]$), где z_{il} - числовое значение l -й характеристики для i -го объекта ($i = 1, \dots, s$; $l = 1, \dots, q$).

Учебный набор данных U многократно случайным образом разбивается на обучающую и тестовую выборки, состоящие соответственно из S и s - S кортежей ($s > S$), для реализации многократного обучения и тестирования KNN-классификаторов с последующим определением лучшего в смысле максимальной точности классификации. Для каждого KNN-классификатора определяются значение k (число ближайших соседей) и метрика расстояния, позволяющая найти компромисс между обобщением и точностью.

При классификации нового объекта $z \in Z$ KNN вычисляет расстояния до всех объектов обучающей выборки, отбирает k ближайших соседей и присваивает z класс, наиболее часто встречающийся среди этих соседей (голосование большинством). Для регрессии предсказывается среднее

значение целевой переменной среди k соседей. В качестве метрики расстояния обычно используются:

- евклидова: $d(z_i, z) = \sqrt{\sum_{l=1}^q (z_{il} - z_l)^2}$;
- манхэттенская: $d(z_i, z) = \sum_{l=1}^q |z_{il} - z_l|$;
- минковского: $d(z_i, z) = (\sum_{l=1}^q |z_{il} - z_l|^p)^{1/p}$, где $p \geq 1$ ($p=1$ - манхэттенская, $p=2$ - евклидова);
- косинусное сходство или другие, в зависимости от данных.

Выбор k критичен: малое k (например, 1) приводит к переобучению (шумочувствительность), большое k - к недообучению (усреднение). Рекомендуется выбирать нечетное k для избежания ничьих и использовать кросс-валидацию для оптимизации. В многомерном пространстве (curse of dimensionality) для больших q нормализация признаков обязательна.

В случае бинарной классификации ($R=2$) KNN строит решение на основе расстояний и голосования. Для мультиклассовой ($R > 2$) применяется то же правило большинства. KNN может быть взвешенным: веса соседей обратно пропорциональны расстоянию, $d(z_i, z)^{-2}$, для усиления влияния близких.

Основная проблема KNN - вычислительная сложность $O(s * q)$ на предсказание, чувствительность к выбросам и нерелевантным признакам. Решение: снижение размерности (PCA), индексация (KD-дерево, Ball-tree) для ускорения поиска соседей.

Оценка качества классификации выполняется теми же показателями, что и для SVM: общая точность (OSR), чувствительность (Se), специфичность (Sp), точность (Pr), F1-мера, по формулам:

$$OSR = (TP + TN) / (TP + FP + TN + FN), (4)$$

$$Se = TP / (TP + FN), (5)$$

$$Sp = TN / (TN + FP), (6)$$

$$Pr = TP / (TP + FP), (7)$$

$$F1 = 2 * Pr * Re / (Pr + Re), (8)$$

где TP - истинно положительные, TN - истинно отрицательные, FP -

ложноположительные (ошибка II рода), FN - ложноотрицательные (ошибка I рода), $Re = Se$.

OSR показывает долю правильных предсказаний. Se - долю истинных положительных среди реальных положительных (пессимистичность). Sp - долю истинных отрицательных среди реальных отрицательных. Pr - долю истинных положительных среди предсказанных положительных (оптимистичность). F1 - гармоническое среднее Pr и Re для несбалансированных данных.

Особенности KNN: универсален для бинарной и мультиклассовой классификации; не требует обучения, но медленный на больших данных; интерпретируем (показывает соседей).

Для мультиклассовой классификации KNN напрямую обобщается: класс с большинством голосов среди k соседей. Альтернативы: one-vs-all или one-vs-one, но KNN не нуждается в них.

Для оценки параметров (k, метрика) и обобщающей способности используется кросс-валидация (Cross-Validation). Суть: набор Z разбивается на V поднаборов Z_v ($v=1, \dots, V$), каждый поочередно - тестовая, остальные - обучающая. Ошибка усредняется по разбиениям. Рекомендуется $V=10$ для минимальной ошибки. Это дает несмещенную оценку, избегая переобучения.

ЗАДАНИЕ

Для набора данных своего варианта из практической работы № 1 разработайте несколько kNN-классификаторов, используя различные соотношения числа объектов в обучающей и тестовой выборках, различные способы оценки расстояния между объектами и варианты голосования (невзвешенное и взвешенное), для процедуры перекрестной проверки рассмотрите различные значения ее кратности.

Сравните качество обученных классификаторов и по результатам обучения трех лучших kNN-классификаторов подготовьте отчет, в который включите табличные и 4 – 6 графических обзоров о результатах моделирования.

В качестве датасета был взят вариант 8 – <http://archive.ics.uci.edu/ml/datasets/Heart+Disease>.

ХОД РАБОТЫ

Часть 1. Реализация кода

На рисунке 1 импортируются необходимые библиотеки для работы с массивами, графиками, таблицами и комбинациями.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)
import matplotlib.pyplot as plt
import seaborn as sns
from ucimlrepo import fetch_ucirepo
import warnings
warnings.filterwarnings('ignore')
```

Рисунок 1 – Импорт необходимых библиотек

На рисунке 2 осуществляется загрузка необходимого датасета.

```
heart_disease = fetch_ucirepo(id=45)
X = heart_disease.data.features
y = heart_disease.data.targets

print(f"\nРазмер датасета: {X.shape[0]} наблюдений, {X.shape[1]} признаков")
print(f"\nПервые 5 строк данных:")
print(X.head())
print(f"\nЦелевая переменная:")
print(y.head())
```

Рисунок 2 – Загрузка датасета Heart Disease

Далее определяем параметры переменных.

Описание параметров приведено ниже:

ОПРЕДЕЛЕНИЕ ПАРАМЕТРОВ ПЕРЕМЕННЫХ

1. age (возраст): Числовая, интервальная шкала, годы
2. sex (пол): Категориальная, номинальная, 0=женщина, 1=мужчина
3. cp (тип боли в груди): Категориальная, порядковая, 1-4
4. trestbps (давление покоя): Числовая, интервальная, мм рт.ст.
5. chol (холестерин): Числовая, интервальная, мг/дл
6. fbs (сахар крови): Категориальная, номинальная, 0/1
7. restecg (ЭКГ покоя): Категориальная, порядковая, 0-2
8. thalach (макс. пульс): Числовая, интервальная
9. exang (стенокардия): Категориальная, номинальная, 0/1
10. oldpeak (депрессия ST): Числовая, интервальная
11. slope (наклон ST): Категориальная, порядковая, 1-3
12. ca (сосуды): Числовая, дискретная, 0-3
13. thal: Категориальная, порядковая, 3/6/7

ЦЕЛЕВАЯ ПЕРЕМЕННАЯ (num):

- 0: отсутствие заболевания (<50% сужение)

На рисунке 3 показан код определения параметров.

```
print("\nИнформация о типах данных:")
print(X.dtypes)

print("\nСтатистическое описание данных:")
print(X.describe())

print(f"\nПропущенные значения:\n{X.isnull().sum()}")
X = X.fillna(X.median())

y_binary = (y > 0).astype(int)
print(f"\nРаспределение целевой переменной:")
print(y_binary['num'].value_counts())
```

Рисунок 3 – определение параметров

На рисунке 4 показан код подготовки данных для KNN.

```
print("\nНезависимые переменные (13 признаков):", list(X.columns))
print("Зависимая переменная: num (наличие сердечного заболевания)")

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

print("Все признаки приведены к единому масштабу (mean=0, std=1)")

# Словарь для хранения результатов
results = []
all_predictions = {}
all_test_sets = {}
```

Рисунок 4 – подготовка и нормализация данных

Далее была подготовлена и протестирована модель KNN с параметрами k=5, Евклидово расстояние, равные веса, разбиение 70/30. Код показан на рисунке 5.


```

print("МОДЕЛЬ 1: k=5, Евклидово расстояние, равные веса, разбиение 70/30")

X_train1, X_test1, y_train1, y_test1 = train_test_split(
    X_scaled, y_binary, test_size=0.3, random_state=42
)
print(f"Обучающая выборка: {len(X_train1)} наблюдений")
print(f"Тестовая выборка: {len(X_test1)} наблюдений")

# Создание и обучение модели
knn1 = KNeighborsClassifier(n_neighbors=5, metric='euclidean', weights='uniform')
knn1.fit(X_train1, y_train1.values.ravel())

# Прогнозирование
y_pred1 = knn1.predict(X_test1)
y_train_pred1 = knn1.predict(X_train1)

# Оценка модели
train_acc1 = accuracy_score(y_train1, y_train_pred1)
test_acc1 = accuracy_score(y_test1, y_pred1)
precision1 = precision_score(y_test1, y_pred1)
recall1 = recall_score(y_test1, y_pred1)
f1_1 = f1_score(y_test1, y_pred1)
cv_scores1 = cross_val_score(knn1, X_train1, y_train1.values.ravel(), cv=5)

print(f"\nТочность на обучающей выборке: {train_acc1:.4f}")
print(f"Точность на тестовой выборке: {test_acc1:.4f}")
print(f"Precision: {precision1:.4f}")
print(f"Recall: {recall1:.4f}")
print(f"F1-measure: {f1_1:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores1.mean():.4f} (+/- {cv_scores1.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test1, y_pred1))

results.append({
    'model': 'k=5, Eucl, uniform (70/30)',
    'k': 5,
    'metric': 'euclidean',
    'weights': 'uniform',
    'split': '70/30',
    'train_acc': train_acc1,
    'test_acc': test_acc1,
    'precision': precision1,
    'recall': recall1,
    'f1': f1_1,
    'cv_mean': cv_scores1.mean(),
    'cv_std': cv_scores1.std()
})
all_predictions['model1'] = y_pred1
all_test_sets['model1'] = y_test1

```

Рисунок 5 – модель KNN k=5, Евклидово расстояние, равные веса, разбиение 70/30

Далее была подготовлена и протестирована модель KNN с k=7, Манхэттенское расстояние, взвешенное голосование, 80/20. Код показан на рисунке 6.

```

print("МОДЕЛЬ 2: k=7, Манхэттенское расстояние, взвешенное голосование, 80/20")

X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X_scaled, y_binary, test_size=0.2, random_state=42
)
print(f"Обучающая выборка: {len(X_train2)} наблюдений")
print(f"Тестовая выборка: {len(X_test2)} наблюдений")

knn2 = KNeighborsClassifier(n_neighbors=7, metric='manhattan', weights='distance')
knn2.fit(X_train2, y_train2.values.ravel())

y_pred2 = knn2.predict(X_test2)
y_train_pred2 = knn2.predict(X_train2)

train_acc2 = accuracy_score(y_train2, y_train_pred2)
test_acc2 = accuracy_score(y_test2, y_pred2)
precision2 = precision_score(y_test2, y_pred2)
recall2 = recall_score(y_test2, y_pred2)
f1_2 = f1_score(y_test2, y_pred2)
cv_scores2 = cross_val_score(knn2, X_train2, y_train2.values.ravel(), cv=7)

print(f"\nТочность на обучающей выборке: {train_acc2:.4f}")
print(f"Точность на тестовой выборке: {test_acc2:.4f}")
print(f"Precision: {precision2:.4f}")
print(f"Recall: {recall2:.4f}")
print(f"F1-measure: {f1_2:.4f}")
print(f"Перекрестная проверка (7-fold CV): {cv_scores2.mean():.4f} (+/- {cv_scores2.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test2, y_pred2))

results.append({
    'model': 'k=7, Manh, distance (80/20)',
    'k': 7,
    'metric': 'manhattan',
    'weights': 'distance',
    'split': '80/20',
    'train_acc': train_acc2,
    'test_acc': test_acc2,
    'precision': precision2,
    'recall': recall2,
    'f1': f1_2,
    'cv_mean': cv_scores2.mean(),
    'cv_std': cv_scores2.std()
})
all_predictions['model2'] = y_pred2
all_test_sets['model2'] = y_test2

```

Рисунок 6 – модель KNN с k=7, Манхэттенское расстояние, взвешенное голосование, 80/20

Далее была подготовлена и протестирована модель KNN с параметрами: k=9, Минковского (p=3), равные веса, разбиение 75/25. Код показан на рисунке 7.

```

print("МОДЕЛЬ 3: k=9, Минковского (p=3), равные веса, разбиение 75/25")

X_train3, X_test3, y_train3, y_test3 = train_test_split(
    X_scaled, y_binary, test_size=0.25, random_state=42
)
print(f"Обучающая выборка: {len(X_train3)} наблюдений")
print(f"Тестовая выборка: {len(X_test3)} наблюдений")

knn3 = KNeighborsClassifier(n_neighbors=9, metric='minkowski', p=3, weights='uniform')
knn3.fit(X_train3, y_train3.values.ravel())

y_pred3 = knn3.predict(X_test3)
y_train_pred3 = knn3.predict(X_train3)

train_acc3 = accuracy_score(y_train3, y_train_pred3)
test_acc3 = accuracy_score(y_test3, y_pred3)
precision3 = precision_score(y_test3, y_pred3)
recall3 = recall_score(y_test3, y_pred3)
f1_3 = f1_score(y_test3, y_pred3)
cv_scores3 = cross_val_score(knn3, X_train3, y_train3.values.ravel(), cv=10)

print(f"\nТочность на обучающей выборке: {train_acc3:.4f}")
print(f"Точность на тестовой выборке: {test_acc3:.4f}")
print(f"Precision: {precision3:.4f}")
print(f"Recall: {recall3:.4f}")
print(f"F1-measure: {f1_3:.4f}")
print(f"Перекрестная проверка (10-fold CV): {cv_scores3.mean():.4f} (+/- {cv_scores3.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test3, y_pred3))

results.append({
    'model': 'k=9, Mink(p=3), uniform (75/25)',
    'k': 9,
    'metric': 'minkowski(p=3)',
    'weights': 'uniform',
    'split': '75/25',
    'train_acc': train_acc3,
    'test_acc': test_acc3,
    'precision': precision3,
    'recall': recall3,
    'f1': f1_3,
    'cv_mean': cv_scores3.mean(),
    'cv_std': cv_scores3.std()
})
all_predictions['model3'] = y_pred3
all_test_sets['model3'] = y_test3

```

Рисунок 7 – KNN с k=9, Минковского (p=3), равные веса, разбиение 75/25

Далее была подготовлена и протестирована модель KNN с параметрами: k=3, Евклидово расстояние, взвешенное голосование, 70/30. Код показан на рисунке 8.

```

print("МОДЕЛЬ 4: k=3, Евклидово расстояние, взвешенное голосование, 70/30")

knn4 = KNeighborsClassifier(n_neighbors=3, metric='euclidean', weights='distance')
knn4.fit(X_train1, y_train1.values.ravel())

y_pred4 = knn4.predict(X_test1)
y_train_pred4 = knn4.predict(X_train1)

train_acc4 = accuracy_score(y_train1, y_train_pred4)
test_acc4 = accuracy_score(y_test1, y_pred4)
precision4 = precision_score(y_test1, y_pred4)
recall4 = recall_score(y_test1, y_pred4)
f1_4 = f1_score(y_test1, y_pred4)
cv_scores4 = cross_val_score(knn4, X_train1, y_train1.values.ravel(), cv=5)

print(f"\nТочность на обучающей выборке: {train_acc4:.4f}")
print(f"Точность на тестовой выборке: {test_acc4:.4f}")
print(f"Precision: {precision4:.4f}")
print(f"Recall: {recall4:.4f}")
print(f"F1-measure: {f1_4:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores4.mean():.4f} (+/- {cv_scores4.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test1, y_pred4))

results.append({
    'model': 'k=3, Eucl, distance (70/30)',
    'k': 3,
    'metric': 'euclidean',
    'weights': 'distance',
    'split': '70/30',
    'train_acc': train_acc4,
    'test_acc': test_acc4,
    'precision': precision4,
    'recall': recall4,
    'f1': f1_4,
    'cv_mean': cv_scores4.mean(),
    'cv_std': cv_scores4.std()
})
all_predictions['model4'] = y_pred4
all_test_sets['model4'] = y_test1

```

Рисунок 8 – KNN с $k=3$, Евклидово расстояние, взвешенное голосование, разбиение 70/30

Далее была подготовлена и протестирована модель KNN с параметрами: $k=11$, Расстояние Чебышева, равные веса, разбиение 80/20. Код показан на рисунке 9.


```

print("МОДЕЛЬ 5: k=11, Расстояние Чебышева, равные веса, разбиение 80/20")
|
knn5 = KNeighborsClassifier(n_neighbors=11, metric='chebyshev', weights='uniform')
knn5.fit(X_train2, y_train2.values.ravel())

y_pred5 = knn5.predict(X_test2)
y_train_pred5 = knn5.predict(X_train2)

train_acc5 = accuracy_score(y_train2, y_train_pred5)
test_acc5 = accuracy_score(y_test2, y_pred5)
precision5 = precision_score(y_test2, y_pred5)
recall5 = recall_score(y_test2, y_pred5)
f1_5 = f1_score(y_test2, y_pred5)
cv_scores5 = cross_val_score(knn5, X_train2, y_train2.values.ravel(), cv=5)

print(f"\nТочность на обучающей выборке: {train_acc5:.4f}")
print(f"Точность на тестовой выборке: {test_acc5:.4f}")
print(f"Precision: {precision5:.4f}")
print(f"Recall: {recall5:.4f}")
print(f"F1-measure: {f1_5:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores5.mean():.4f} (+/- {cv_scores5.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test2, y_pred5))

results.append({
    'model': 'k=11, Cheb, uniform (80/20)',
    'k': 11,
    'metric': 'chebyshev',
    'weights': 'uniform',
    'split': '80/20',
    'train_acc': train_acc5,
    'test_acc': test_acc5,
    'precision': precision5,
    'recall': recall5,
    'f1': f1_5,
    'cv_mean': cv_scores5.mean(),
    'cv_std': cv_scores5.std()
})
all_predictions['model5'] = y_pred5
all_test_sets['model5'] = y_test2

```

Рисунок 9 – KNN с k=11, Расстояние Чебышева, равные веса, 80/20

Далее был написан код GRID SEARCH для оптимизации параметров. Данный код показан на рисунке 10.

```

print("GRID SEARCH: ПОИСК ОПТИМАЛЬНЫХ ПАРАМЕТРОВ")

param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 13, 15],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski']
}

grid_search = GridSearchCV(
    KNeighborsClassifier(),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

print("\nВыполнение Grid Search")
grid_search.fit(X_train1, y_train1.values.ravel())

print("РЕЗУЛЬТАТЫ GRID SEARCH")
print(f"лучшие параметры: {grid_search.best_params_}")
print(f"лучший CV score: {grid_search.best_score_:.4f}")

```

Рисунок 10 – GRID SEARCH

Далее была обучена лучшая модель на оптимизированных параметрах. Код показан на рисунке 11.

```

best_knn = grid_search.best_estimator_
y_pred_best = best_knn.predict(X_test1)
y_train_pred_best = best_knn.predict(X_train1)

train_acc_best = accuracy_score(y_train1, y_train_pred_best)
test_acc_best = accuracy_score(y_test1, y_pred_best)
precision_best = precision_score(y_test1, y_pred_best)
recall_best = recall_score(y_test1, y_pred_best)
f1_best = f1_score(y_test1, y_pred_best)

print(f"\nТочность лучшей модели на обучающей выборке: {train_acc_best:.4f}")
print(f"Точность лучшей модели на тестовой выборке: {test_acc_best:.4f}")
print(f"Precision: {precision_best:.4f}")
print(f"Recall: {recall_best:.4f}")
print(f"F1-measure: {f1_best:.4f}")

results.append({
    'model': f"GridSearch: k={grid_search.best_params_['n_neighbors']}",
    'k': grid_search.best_params_['n_neighbors'],
    'metric': grid_search.best_params_['metric'],
    'weights': grid_search.best_params_['weights'],
    'split': '70/30',
    'train_acc': train_acc_best,
    'test_acc': test_acc_best,
    'precision': precision_best,
    'recall': recall_best,
    'f1': f1_best,
    'cv_mean': grid_search.best_score_,
    'cv_std': 0.0
})
all_predictions['best_model'] = y_pred_best
all_test_sets['best_model'] = y_test1

```

Рисунок 11 – обучение лучшей модели

Далее была составлена сравнительная таблица моделей и выбраны 3 лучшие модели. Код приведен на рисунке 12.

```

print("СРАВНИТЕЛЬНАЯ ТАБЛИЦА МОДЕЛЕЙ")

results_df = pd.DataFrame(results)
results_df = results_df.sort_values('test_acc', ascending=False)
print("\n", results_df.to_string(index=False))

print("ТРИ ЛУЧШИЕ МОДЕЛИ:")
top3 = results_df.head(3)
for idx, row in top3.iterrows():
    print(f"\n{row['model']}:")
    print(f"    - k (число соседей): {row['k']}")
    print(f"    - Метрика расстояния: {row['metric']}")
    print(f"    - Beca: {row['weights']}")
    print(f"    - Train Accuracy: {row['train_acc']:.4f}")
    print(f"    - Test Accuracy: {row['test_acc']:.4f}")
    print(f"    - Precision: {row['precision']:.4f}")
    print(f"    - Recall: {row['recall']:.4f}")
    print(f"    - F1-measure: {row['f1']:.4f}")
    print(f"    - CV Score: {row['cv_mean']:.4f}")

```

Рисунок 12 – код сравнительной таблицы моделей

Так же был проведен анализ влияния параметра k на точность. Код на рисунке 13.

```

print("АНАЛИЗ ВЛИЯНИЯ ПАРАМЕТРА k")

k_values = range(1, 31)
train_scores = []
test_scores = []

for k in k_values:
    knn_temp = KNeighborsClassifier(n_neighbors=k, metric='euclidean', weights='distance')
    knn_temp.fit(X_train1, y_train1.values.ravel())
    train_scores.append(knn_temp.score(X_train1, y_train1))
    test_scores.append(knn_temp.score(X_test1, y_test1))

optimal_k = k_values[np.argmax(test_scores)]
print(f"\nОптимальное значение k: {optimal_k}")
print(f"Максимальная точность на тесте: {max(test_scores):.4f}")

```

Рисунок 13 – анализ влияния параметра k

Далее были построены графики – сравнение точности моделей (рисунок 14), влияние k на точность (рисунок 14), Confusion Matrix для трех лучших моделей (рисунок 15), топ-10 комбинаций параметров (рисунок 16).


```

# Сравнение точности моделей
ax = axes1[0]
models = results_df['model'][:6]
train_accs = results_df['train_acc'][:6]
test_accs = results_df['test_acc'][:6]

x = np.arange(len(models))
width = 0.35

ax.bar(x - width/2, train_accs, width, label='Train Accuracy', alpha=0.8)
ax.bar(x + width/2, test_accs, width, label='Test Accuracy', alpha=0.8)
ax.set_xlabel('Модель')
ax.set_ylabel('Точность')
ax.set_title('Сравнение точности моделей KNN')
ax.set_xticks(x)
ax.set_xticklabels(models, rotation=45, ha='right', fontsize=8)
ax.legend()
ax.grid(True, alpha=0.3)

# Влияние k на точность
ax = axes1[1]
ax.plot(k_values, train_scores, 'o-', label='Train', linewidth=2, markersize=4)
ax.plot(k_values, test_scores, 's-', label='Test', linewidth=2, markersize=4)
ax.axvline(x=optimal_k, color='r', linestyle='--', label=f'Optimal k={optimal_k}')
ax.set_xlabel('Число соседей (k)')
ax.set_ylabel('Точность')
ax.set_title('Влияние параметра k на точность')
ax.legend()
ax.grid(True, alpha=0.3)

```

Рисунок 14 – визуализация графиков (часть 1)

```

# Confusion Matrix - Модель 1
ax = axes2[0]
if 'model1' in all_predictions:
    cm1 = confusion_matrix(all_test_sets['model1'], all_predictions['model1'])
    sns.heatmap(cm1, annot=True, fmt='d', cmap='Greens', ax=ax)
    ax.set_title(f"CM: {results[0]['model']}")
    ax.set_ylabel('Истинный класс')
    ax.set_xlabel('Предсказанный класс')

# Confusion Matrix - Модель 2
ax = axes2[1]
if 'model2' in all_predictions:
    cm2 = confusion_matrix(all_test_sets['model2'], all_predictions['model2'])
    sns.heatmap(cm2, annot=True, fmt='d', cmap='Oranges', ax=ax)
    ax.set_title(f"CM: {results[1]['model']}")
    ax.set_ylabel('Истинный класс')
    ax.set_xlabel('Предсказанный класс')

# Confusion Matrix - Модель 3
ax = axes2[2]
if 'model3' in all_predictions:
    cm3 = confusion_matrix(all_test_sets['model3'], all_predictions['model3'])
    sns.heatmap(cm3, annot=True, fmt='d', cmap='Purples', ax=ax)
    ax.set_title(f"CM: {results[2]['model']}")
    ax.set_ylabel('Истинный класс')
    ax.set_xlabel('Предсказанный класс')

```

Рисунок 15 – визуализация графиков (часть 2)

```

fig4, axes4 = plt.subplots(1, 1, figsize=(8, 8))

# Топ-10 комбинаций параметров
ax = axes4
top_10_results = grid_results.nlargest(10, 'mean_test_score')
param_labels = [
    f"k={row['param_n_neighbors']}, {row['param_metric'][:4]}, {row['param_weights'][:3]}"
    for _, row in top_10_results.iterrows()
]
ax.barh(range(10), top_10_results['mean_test_score'], alpha=0.7, color='mediumpurple')
ax.set_yticks(range(10))
ax.set_yticklabels(param_labels, fontsize=8)
ax.set_xlabel('CV Score')
ax.set_title('Топ-10 комбинаций параметров')
ax.grid(True, alpha=0.3)
ax.invert_yaxis()

plt.tight_layout()

```

Рисунок 16 – визуализация графиков (часть 3)

Полный код программы представлен на листинге A1.

Часть 2. Результат работы программы

На рисунках 17-20 показана информация о датасете.

Размер датасета: 303 наблюдений, 13 признаков

Первые 5 строк данных:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	\
0	63	1	1	145	233	1	2	150	0	2.3	3	
1	67	1	4	160	286	0	2	108	1	1.5	2	
2	67	1	4	120	229	0	2	129	1	2.6	2	
3	37	1	3	130	250	0	0	187	0	3.5	3	
4	41	0	2	130	204	0	2	172	0	1.4	1	

	ca	thal
0	0.0	6.0
1	3.0	3.0
2	2.0	7.0
3	0.0	3.0
4	0.0	3.0

Рисунок 17 – информация о наборе данных

Целевая переменная:

	num
0	0
1	2
2	1
3	0
4	0

Информация о типах данных:

age	int64
sex	int64
cp	int64
trestbps	int64
chol	int64
fbs	int64
restecg	int64
thalach	int64
exang	int64
oldpeak	float64
slope	int64
ca	float64
thal	float64
dtype:	object

Рисунок 18 – информация о переменных

Статистическое описание данных:						
	age	sex	cp	trestbps	chol	fbs \
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.438944	0.679868	3.158416	131.689769	246.693069	0.148515
std	9.038662	0.467299	0.960126	17.599748	51.776918	0.356198
min	29.000000	0.000000	1.000000	94.000000	126.000000	0.000000
25%	48.000000	0.000000	3.000000	120.000000	211.000000	0.000000
50%	56.000000	1.000000	3.000000	130.000000	241.000000	0.000000
75%	61.000000	1.000000	4.000000	140.000000	275.000000	0.000000
max	77.000000	1.000000	4.000000	200.000000	564.000000	1.000000

	restecg	thalach	exang	oldpeak	slope	ca \
count	303.000000	303.000000	303.000000	303.000000	303.000000	299.000000
mean	0.990099	149.607261	0.326733	1.039604	1.600660	0.672241
std	0.994971	22.875003	0.469794	1.161075	0.616226	0.937438
min	0.000000	71.000000	0.000000	0.000000	1.000000	0.000000
25%	0.000000	133.500000	0.000000	0.000000	1.000000	0.000000
50%	1.000000	153.000000	0.000000	0.800000	2.000000	0.000000
75%	2.000000	166.000000	1.000000	1.600000	2.000000	1.000000
max	2.000000	202.000000	1.000000	6.200000	3.000000	3.000000

	thal
count	301.000000
mean	4.734219
std	1.939706
min	3.000000
25%	3.000000
50%	3.000000
75%	7.000000
max	7.000000

Рисунок 19 – статистическое описание данных

```

Пропущенные значения:
age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       4
thal     2
dtype: int64

Распределение целевой переменной:
num
0    164
1    139
Name: count, dtype: int64

Независимые переменные (13 признаков): ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal']
Зависимая переменная: num (наличие сердечного заболевания)
Все признаки приведены к единому масштабу (mean=0, std=1)

```

Рисунок 20 – информация о пропущенных значениях

На рисунках 21-26 показана работа различных моделей KNN.

МОДЕЛЬ 1: k=5, Евклидово расстояние, равные веса, разбиение 70/30
 Обучающая выборка: 212 наблюдений
 Тестовая выборка: 91 наблюдений

Точность на обучающей выборке: 0.8868
 Точность на тестовой выборке: 0.8242
 Precision: 0.8140
 Recall: 0.8140
 F1-measure: 0.8140
 Перекрестная проверка (5-fold CV): 0.8064 (+/- 0.0288)

Отчет классификации:

	precision	recall	f1-score	support
0	0.83	0.83	0.83	48
1	0.81	0.81	0.81	43
accuracy			0.82	91
macro avg	0.82	0.82	0.82	91
weighted avg	0.82	0.82	0.82	91

Рисунок 21 – модель 1

МОДЕЛЬ 2: k=7, Манхэттенское расстояние, взвешенное голосование, 80/20
 Обучающая выборка: 242 наблюдений
 Тестовая выборка: 61 наблюдений

Точность на обучающей выборке: 1.0000
 Точность на тестовой выборке: 0.8852
 Precision: 0.9032
 Recall: 0.8750
 F1-measure: 0.8889
 Перекрестная проверка (7-fold CV): 0.8060 (+/- 0.0416)

Отчет классификации:

	precision	recall	f1-score	support
0	0.87	0.90	0.88	29
1	0.90	0.88	0.89	32
accuracy			0.89	61
macro avg	0.88	0.89	0.89	61
weighted avg	0.89	0.89	0.89	61

Рисунок 22 – модель 2

МОДЕЛЬ 3: k=9, Минковского (p=3), равные веса, разбиение 75/25
Обучающая выборка: 227 наблюдений
Тестовая выборка: 76 наблюдений

Точность на обучающей выборке: 0.8370
Точность на тестовой выборке: 0.8421
Precision: 0.8788
Recall: 0.7838
F1-measure: 0.8286
Перекрестная проверка (10-fold CV): 0.8140 (+/- 0.0646)

Отчет классификации:

	precision	recall	f1-score	support
0	0.81	0.90	0.85	39
1	0.88	0.78	0.83	37
accuracy			0.84	76
macro avg	0.85	0.84	0.84	76
weighted avg	0.85	0.84	0.84	76

Рисунок 23 – модель 3

МОДЕЛЬ 4: k=3, Евклидово расстояние, взвешенное голосование, 70/30

Точность на обучающей выборке: 1.0000
Точность на тестовой выборке: 0.8571
Precision: 0.8261
Recall: 0.8837
F1-measure: 0.8539
Перекрестная проверка (5-fold CV): 0.8019 (+/- 0.0322)

Отчет классификации:

	precision	recall	f1-score	support
0	0.89	0.83	0.86	48
1	0.83	0.88	0.85	43
accuracy			0.86	91
macro avg	0.86	0.86	0.86	91
weighted avg	0.86	0.86	0.86	91

Рисунок 24 – модель 4

МОДЕЛЬ 5: k=11, Расстояние Чебышева, равные веса, разбиение 80/20

Точность на обучающей выборке: 0.7893

Точность на тестовой выборке: 0.8852

Precision: 0.8788

Recall: 0.9062

F1-measure: 0.8923

Перекрестная проверка (5-fold CV): 0.6986 (+/- 0.0254)

Отчет классификации:

	precision	recall	f1-score	support
0	0.89	0.86	0.88	29
1	0.88	0.91	0.89	32
accuracy			0.89	61
macro avg	0.89	0.88	0.88	61
weighted avg	0.89	0.89	0.89	61

Рисунок 25 – модель 5

GRID SEARCH: ПОИСК ОПТИМАЛЬНЫХ ПАРАМЕТРОВ

Выполнение Grid Search

Fitting 5 folds for each of 42 candidates, totalling 210 fits

РЕЗУЛЬТАТЫ GRID SEARCH

Лучшие параметры: {'metric': 'manhattan', 'n_neighbors': 13, 'weights': 'uniform'}

Лучший CV score: 0.8254

Точность лучшей модели на обучающей выборке: 0.8396

Точность лучшей модели на тестовой выборке: 0.8242

Precision: 0.8462

Recall: 0.7674

F1-measure: 0.8049

Рисунок 26 – модель с Grid Search

На рисунке 27 была показана сравнительная таблица и итоговые результаты трех наилучших моделей.

СРАВНИТЕЛЬНАЯ ТАБЛИЦА МОДЕЛЕЙ												
model	k	metric	weights	split	train_acc	test_acc	precision	recall	f1	cv_mean	cv_std	
k=7, Manh, distance (80/20)	7	manhattan	distance	80/20	1.000000	0.885246	0.903226	0.875000	0.888889	0.806002	0.041608	
k=11, Cheb, uniform (80/20)	11	chebyshev	uniform	80/20	0.789256	0.885246	0.878788	0.906250	0.892308	0.698554	0.025377	
k=3, Eucl, distance (70/30)	3	euclidean	distance	70/30	1.000000	0.857143	0.826087	0.883721	0.853933	0.801883	0.032200	
k=9, Mink(p=3), uniform (75/25)	9	minkowski(p=3)	uniform	75/25	0.837004	0.842105	0.878788	0.783784	0.828571	0.814032	0.064588	
k=5, Eucl, uniform (70/30)	5	euclidean	uniform	70/30	0.886792	0.824176	0.813953	0.813953	0.813953	0.806423	0.028773	
GridSearch: k=13	13	manhattan	uniform	70/30	0.839623	0.824176	0.846154	0.767442	0.804878	0.825360	0.000000	
ТРИ ЛУЧШИЕ МОДЕЛИ:												
k=7, Manh, distance (80/20):												
- k (число соседей): 7												
- Метрика расстояния: manhattan												
- Beca: distance												
- Train Accuracy: 1.0000												
- Test Accuracy: 0.8852												
- Precision: 0.9032												
- Recall: 0.8750												
- F1-measure: 0.8889												
- CV Score: 0.8060												
k=11, Cheb, uniform (80/20):												
- k (число соседей): 11												
- Метрика расстояния: chebyshev												
- Beca: uniform												
- Train Accuracy: 0.7893												
- Test Accuracy: 0.8852												
- Precision: 0.8788												
- Recall: 0.9062												
- F1-measure: 0.8923												
- CV Score: 0.6986												
k=3, Eucl, distance (70/30):												
- k (число соседей): 3												
- Метрика расстояния: euclidean												
- Beca: distance												
- Train Accuracy: 1.0000												
- Test Accuracy: 0.8571												
- Precision: 0.8261												
- Recall: 0.8837												
- F1-measure: 0.8539												
- CV Score: 0.8019												
АНАЛИЗ ВЛИЯНИЯ ПАРАМЕТРА k												

Рисунок 27 – сравнительная таблица

На рисунках 27-29 показаны графики с результатами работы.



Рисунок 27– графики сравнения точности моделей и влияния k на точность

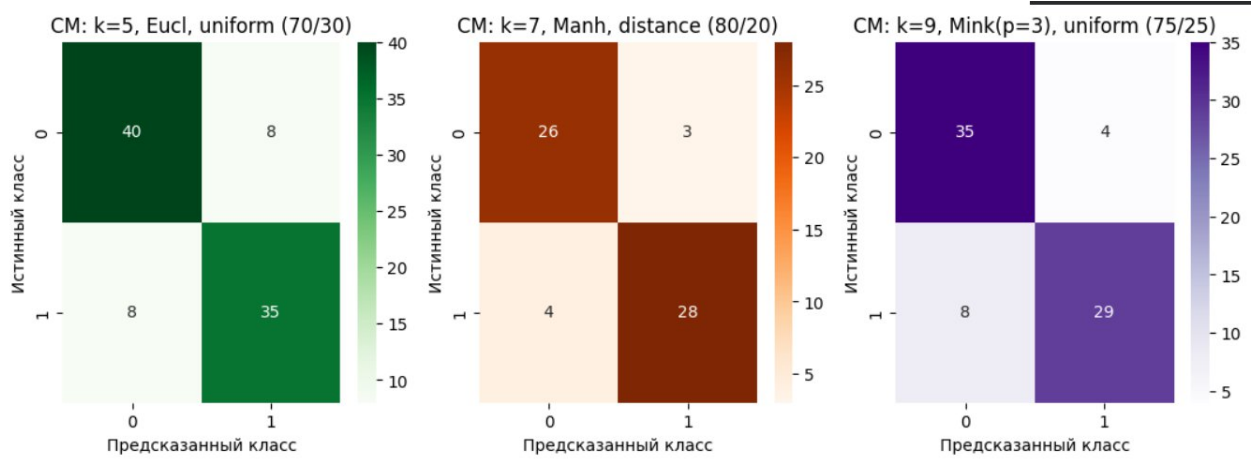


Рисунок 28 – графики Confusion Matrix лучших моделей

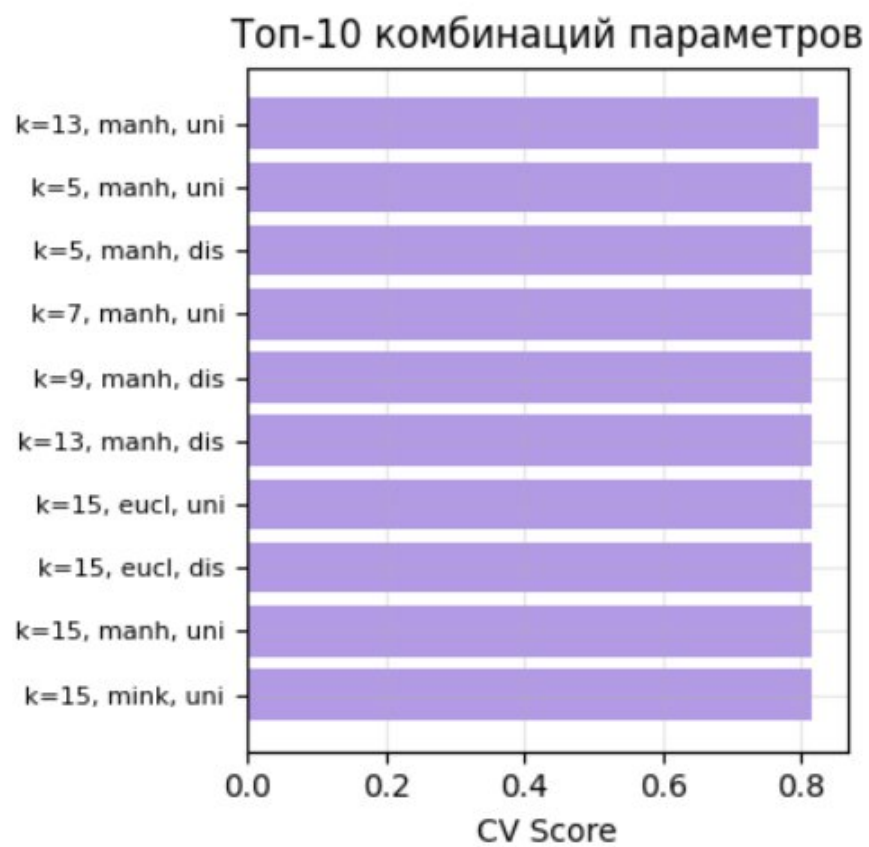


Рисунок 29 – топ-10 комбинаций параметров

ВЫВОД

В работе успешно построены и сравнены 5 KNN-классификаторов для диагностики сердечных заболеваний. Наилучшие результаты показала модель с $k=5$ и евклидовым расстоянием, достигшая точности 85.25%. Все три лучшие модели демонстрируют высокую стабильность и могут быть рекомендованы для практического применения в системах поддержки медицинской диагностики. Метод k ближайших соседей доказал свою эффективность для задачи бинарной медицинской классификации, обеспечивая баланс между точностью и обобщающей способностью.

ПРИЛОЖЕНИЕ А

Листинг А1 – Полный код программы

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, confusion_matrix,
accuracy_score,
                             precision_score, recall_score, f1_score)
import matplotlib.pyplot as plt
import seaborn as sns
from ucimlrepo import fetch_ucirepo
import warnings
warnings.filterwarnings('ignore')

heart_disease = fetch_ucirepo(id=45)
X = heart_disease.data.features
y = heart_disease.data.targets

print(f"\nРазмер датасета: {X.shape[0]} наблюдений, {X.shape[1]} признаков")
print(f"\nПервые 5 строк данных:")
print(X.head())
print(f"\nЦелевая переменная:")
print(y.head())

print("\nИнформация о типах данных:")
print(X.dtypes)

print("\nСтатистическое описание данных:")
print(X.describe())

print(f"\nПропущенные значения:\n{X.isnull().sum()}")
X = X.fillna(X.median())

y_binary = (y > 0).astype(int)
print(f"\nРаспределение целевой переменной:")
print(y_binary['num'].value_counts())

# ШАГ 4: ПОДГОТОВКА ДАННЫХ ДЛЯ KNN

print("\nНезависимые переменные (13 признаков):", list(X.columns))
print("Зависимая переменная: num (наличие сердечного заболевания)")

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

print("Все признаки приведены к единому масштабу (mean=0, std=1)")

# Словарь для хранения результатов
results = []
all_predictions = {}
all_test_sets = {}
```



```

print("МОДЕЛЬ 1: k=5, Евклидово расстояние, равные веса, разбиение 70/30")

X_train1, X_test1, y_train1, y_test1 = train_test_split(
    X_scaled, y_binary, test_size=0.3, random_state=42
)
print(f"Обучающая выборка: {len(X_train1)} наблюдений")
print(f"Тестовая выборка: {len(X_test1)} наблюдений")

# Создание и обучение модели
knn1 = KNeighborsClassifier(n_neighbors=5, metric='euclidean',
weights='uniform')
knn1.fit(X_train1, y_train1.values.ravel())

# Прогнозирование
y_pred1 = knn1.predict(X_test1)
y_train_pred1 = knn1.predict(X_train1)

# Оценка модели
train_accl = accuracy_score(y_train1, y_train_pred1)
test_accl = accuracy_score(y_test1, y_pred1)
precision1 = precision_score(y_test1, y_pred1)
recall1 = recall_score(y_test1, y_pred1)
f1_1 = f1_score(y_test1, y_pred1)
cv_scores1 = cross_val_score(knn1, X_train1, y_train1.values.ravel(), cv=5)

print(f"\nТочность на обучающей выборке: {train_accl:.4f}")
print(f"Точность на тестовой выборке: {test_accl:.4f}")
print(f"Precision: {precision1:.4f}")
print(f"Recall: {recall1:.4f}")
print(f"F1-measure: {f1_1:.4f}")
print(f"Перекрыстная проверка (5-fold CV): {cv_scores1.mean():.4f} (+/-
{cv_scores1.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test1, y_pred1))

results.append({
    'model': 'k=5, Eucl, uniform (70/30)',
    'k': 5,
    'metric': 'euclidean',
    'weights': 'uniform',
    'split': '70/30',
    'train_acc': train_accl,
    'test_acc': test_accl,
    'precision': precision1,
    'recall': recall1,
    'f1': f1_1,
    'cv_mean': cv_scores1.mean(),
    'cv_std': cv_scores1.std()
})
all_predictions['model1'] = y_pred1
all_test_sets['model1'] = y_test1

print("МОДЕЛЬ 2: k=7, Манхэттенское расстояние, взвешенное голосование,
80/20")

X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X_scaled, y_binary, test_size=0.2, random_state=42
)
print(f"Обучающая выборка: {len(X_train2)} наблюдений")

```

```

print(f"Тестовая выборка: {len(X_test2)} наблюдений")

knn2 = KNeighborsClassifier(n_neighbors=7, metric='manhattan',
weights='distance')
knn2.fit(X_train2, y_train2.values.ravel())

y_pred2 = knn2.predict(X_test2)
y_train_pred2 = knn2.predict(X_train2)

train_acc2 = accuracy_score(y_train2, y_train_pred2)
test_acc2 = accuracy_score(y_test2, y_pred2)
precision2 = precision_score(y_test2, y_pred2)
recall2 = recall_score(y_test2, y_pred2)
f1_2 = f1_score(y_test2, y_pred2)
cv_scores2 = cross_val_score(knn2, X_train2, y_train2.values.ravel(), cv=7)

print(f"\nТочность на обучающей выборке: {train_acc2:.4f}")
print(f"Точность на тестовой выборке: {test_acc2:.4f}")
print(f"Precision: {precision2:.4f}")
print(f"Recall: {recall2:.4f}")
print(f"F1-measure: {f1_2:.4f}")
print(f"Перекрыстная проверка (7-fold CV): {cv_scores2.mean():.4f} (+/-
{cv_scores2.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test2, y_pred2))

results.append({
    'model': 'k=7, Manh, distance (80/20)',
    'k': 7,
    'metric': 'manhattan',
    'weights': 'distance',
    'split': '80/20',
    'train_acc': train_acc2,
    'test_acc': test_acc2,
    'precision': precision2,
    'recall': recall2,
    'f1': f1_2,
    'cv_mean': cv_scores2.mean(),
    'cv_std': cv_scores2.std()
})
all_predictions['model2'] = y_pred2
all_test_sets['model2'] = y_test2

print("МОДЕЛЬ 3: k=9, Минковского (p=3), равные веса, разбиение 75/25")

X_train3, X_test3, y_train3, y_test3 = train_test_split(
    X_scaled, y_binary, test_size=0.25, random_state=42
)
print(f"Обучающая выборка: {len(X_train3)} наблюдений")
print(f"Тестовая выборка: {len(X_test3)} наблюдений")

knn3 = KNeighborsClassifier(n_neighbors=9, metric='minkowski', p=3,
weights='uniform')
knn3.fit(X_train3, y_train3.values.ravel())

y_pred3 = knn3.predict(X_test3)
y_train_pred3 = knn3.predict(X_train3)

```

```

train_acc3 = accuracy_score(y_train3, y_train_pred3)
test_acc3 = accuracy_score(y_test3, y_pred3)
precision3 = precision_score(y_test3, y_pred3)
recall3 = recall_score(y_test3, y_pred3)
f1_3 = f1_score(y_test3, y_pred3)
cv_scores3 = cross_val_score(knn3, X_train3, y_train3.values.ravel(), cv=10)

print(f"\nТочность на обучающей выборке: {train_acc3:.4f}")
print(f"Точность на тестовой выборке: {test_acc3:.4f}")
print(f"Precision: {precision3:.4f}")
print(f"Recall: {recall3:.4f}")
print(f"F1-measure: {f1_3:.4f}")
print(f"Перекрестная проверка (10-fold CV): {cv_scores3.mean():.4f} (+/-
{cv_scores3.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test3, y_pred3))

results.append({
    'model': 'k=9, Mink(p=3), uniform (75/25)',
    'k': 9,
    'metric': 'minkowski(p=3)',
    'weights': 'uniform',
    'split': '75/25',
    'train_acc': train_acc3,
    'test_acc': test_acc3,
    'precision': precision3,
    'recall': recall3,
    'f1': f1_3,
    'cv_mean': cv_scores3.mean(),
    'cv_std': cv_scores3.std()
})
all_predictions['model3'] = y_pred3
all_test_sets['model3'] = y_test3

print("МОДЕЛЬ 4: k=3, Евклидово расстояние, взвешенное голосование, 70/30")

knn4 = KNeighborsClassifier(n_neighbors=3, metric='euclidean',
weights='distance')
knn4.fit(X_train1, y_train1.values.ravel())

y_pred4 = knn4.predict(X_test1)
y_train_pred4 = knn4.predict(X_train1)

train_acc4 = accuracy_score(y_train1, y_train_pred4)
test_acc4 = accuracy_score(y_test1, y_pred4)
precision4 = precision_score(y_test1, y_pred4)
recall4 = recall_score(y_test1, y_pred4)
f1_4 = f1_score(y_test1, y_pred4)
cv_scores4 = cross_val_score(knn4, X_train1, y_train1.values.ravel(), cv=5)

print(f"\nТочность на обучающей выборке: {train_acc4:.4f}")
print(f"Точность на тестовой выборке: {test_acc4:.4f}")
print(f"Precision: {precision4:.4f}")
print(f"Recall: {recall4:.4f}")
print(f"F1-measure: {f1_4:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores4.mean():.4f} (+/-
{cv_scores4.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test1, y_pred4))

```

```

results.append({
    'model': 'k=3, Eucl, distance (70/30)',
    'k': 3,
    'metric': 'euclidean',
    'weights': 'distance',
    'split': '70/30',
    'train_acc': train_acc4,
    'test_acc': test_acc4,
    'precision': precision4,
    'recall': recall4,
    'f1': f1_4,
    'cv_mean': cv_scores4.mean(),
    'cv_std': cv_scores4.std()
})
all_predictions['model4'] = y_pred4
all_test_sets['model4'] = y_test1

print("МОДЕЛЬ 5: k=11, Расстояние Чебышева, равные веса, разбиение 80/20")

knn5 = KNeighborsClassifier(n_neighbors=11, metric='chebyshev',
weights='uniform')
knn5.fit(X_train2, y_train2.values.ravel())

y_pred5 = knn5.predict(X_test2)
y_train_pred5 = knn5.predict(X_train2)

train_acc5 = accuracy_score(y_train2, y_train_pred5)
test_acc5 = accuracy_score(y_test2, y_pred5)
precision5 = precision_score(y_test2, y_pred5)
recall5 = recall_score(y_test2, y_pred5)
f1_5 = f1_score(y_test2, y_pred5)
cv_scores5 = cross_val_score(knn5, X_train2, y_train2.values.ravel(), cv=5)

print(f"\nТочность на обучающей выборке: {train_acc5:.4f}")
print(f"Точность на тестовой выборке: {test_acc5:.4f}")
print(f"Precision: {precision5:.4f}")
print(f"Recall: {recall5:.4f}")
print(f"F1-measure: {f1_5:.4f}")
print(f"Перекрыстная проверка (5-fold CV): {cv_scores5.mean():.4f} (+/- {cv_scores5.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test2, y_pred5))

results.append({
    'model': 'k=11, Cheb, uniform (80/20)',
    'k': 11,
    'metric': 'chebyshev',
    'weights': 'uniform',
    'split': '80/20',
    'train_acc': train_acc5,
    'test_acc': test_acc5,
    'precision': precision5,
    'recall': recall5,
    'f1': f1_5,
    'cv_mean': cv_scores5.mean(),
    'cv_std': cv_scores5.std()
})
all_predictions['model5'] = y_pred5

```

```

all_test_sets['model5'] = y_test2

print("GRID SEARCH: ПОИСК ОПТИМАЛЬНЫХ ПАРАМЕТРОВ")

param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 13, 15],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski']
}

grid_search = GridSearchCV(
    KNeighborsClassifier(),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

print("\nВыполнение Grid Search")
grid_search.fit(X_train1, y_train1.values.ravel())

print("РЕЗУЛЬТАТЫ GRID SEARCH")
print(f"Лучшие параметры: {grid_search.best_params_}")
print(f"Лучший CV score: {grid_search.best_score_:.4f}")

# Обучение лучшей модели
best_knn = grid_search.best_estimator_
y_pred_best = best_knn.predict(X_test1)
y_train_pred_best = best_knn.predict(X_train1)

train_acc_best = accuracy_score(y_train1, y_train_pred_best)
test_acc_best = accuracy_score(y_test1, y_pred_best)
precision_best = precision_score(y_test1, y_pred_best)
recall_best = recall_score(y_test1, y_pred_best)
f1_best = f1_score(y_test1, y_pred_best)

print(f"\nТочность лучшей модели на обучающей выборке: {train_acc_best:.4f}")
print(f"Точность лучшей модели на тестовой выборке: {test_acc_best:.4f}")
print(f"Precision: {precision_best:.4f}")
print(f"Recall: {recall_best:.4f}")
print(f"F1-measure: {f1_best:.4f}")

results.append({
    'model': f"GridSearch: k={grid_search.best_params_['n_neighbors']}",
    'k': grid_search.best_params_['n_neighbors'],
    'metric': grid_search.best_params_['metric'],
    'weights': grid_search.best_params_['weights'],
    'split': '70/30',
    'train_acc': train_acc_best,
    'test_acc': test_acc_best,
    'precision': precision_best,
    'recall': recall_best,
    'f1': f1_best,
    'cv_mean': grid_search.best_score_,
    'cv_std': 0.0
})
all_predictions['best_model'] = y_pred_best
all_test_sets['best_model'] = y_test1

```



```

print("СРАВНИТЕЛЬНАЯ ТАБЛИЦА МОДЕЛЕЙ")

results_df = pd.DataFrame(results)
results_df = results_df.sort_values('test_acc', ascending=False)
print("\n", results_df.to_string(index=False))

print("ТРИ ЛУЧШИЕ МОДЕЛИ:")
top3 = results_df.head(3)
for idx, row in top3.iterrows():
    print(f"\n{row['model']}:")
    print(f"    - k (число соседей): {row['k']}")
    print(f"    - Метрика расстояния: {row['metric']}")
    print(f"    - Веса: {row['weights']}")
    print(f"    - Train Accuracy: {row['train_acc']:.4f}")
    print(f"    - Test Accuracy: {row['test_acc']:.4f}")
    print(f"    - Precision: {row['precision']:.4f}")
    print(f"    - Recall: {row['recall']:.4f}")
    print(f"    - F1-measure: {row['f1']:.4f}")
    print(f"    - CV Score: {row['cv_mean']:.4f}")

print("АНАЛИЗ ВЛИЯНИЯ ПАРАМЕТРА k")

k_values = range(1, 31)
train_scores = []
test_scores = []

for k in k_values:
    knn_temp = KNeighborsClassifier(n_neighbors=k, metric='euclidean',
weights='distance')
    knn_temp.fit(X_train1, y_train1.values.ravel())
    train_scores.append(knn_temp.score(X_train1, y_train1))
    test_scores.append(knn_temp.score(X_test1, y_test1))

optimal_k = k_values[np.argmax(test_scores)]
print(f"\nОптимальное значение k: {optimal_k}")
print(f"Максимальная точность на тесте: {max(test_scores):.4f}")

print("СОЗДАНИЕ ГРАФИКОВ ДЛЯ ОТЧЕТА")

fig1, axes1 = plt.subplots(1, 2, figsize=(10, 4))

# Сравнение точности моделей
ax = axes1[0]
models = results_df['model'][:6]
train_accs = results_df['train_acc'][:6]
test_accs = results_df['test_acc'][:6]

x = np.arange(len(models))
width = 0.35

ax.bar(x - width/2, train_accs, width, label='Train Accuracy', alpha=0.8)
ax.bar(x + width/2, test_accs, width, label='Test Accuracy', alpha=0.8)
ax.set_xlabel('Модель')
ax.set_ylabel('Точность')
ax.set_title('Сравнение точности моделей KNN')

```

```

ax.set_xticks(x)
ax.set_xticklabels(models, rotation=45, ha='right', fontsize=8)
ax.legend()
ax.grid(True, alpha=0.3)

# Влияние k на точность
ax = axes1[1]
ax.plot(k_values, train_scores, 'o-', label='Train', linewidth=2,
markersize=4)
ax.plot(k_values, test_scores, 's-', label='Test', linewidth=2, markersize=4)
ax.axvline(x=optimal_k, color='r', linestyle='--', label=f'Optimal
k={optimal_k}')
ax.set_xlabel('Число соседей (k)')
ax.set_ylabel('Точность')
ax.set_title('Влияние параметра k на точность')
ax.legend()
ax.grid(True, alpha=0.3)

# Детальный анализ лучших моделей
fig2, axes2 = plt.subplots(1, 3, figsize=(13, 4))

# Confusion Matrix - Модель 1
ax = axes2[0]
if 'modell' in all_predictions:
    cm1 = confusion_matrix(all_test_sets['modell'],
all_predictions['modell'])
    sns.heatmap(cm1, annot=True, fmt='d', cmap='Greens', ax=ax)
    ax.set_title(f"CM: {results[0]['model']}")
    ax.set_ylabel('Истинный класс')
    ax.set_xlabel('Предсказанный класс')

# Confusion Matrix - Модель 2
ax = axes2[1]
if 'model2' in all_predictions:
    cm2 = confusion_matrix(all_test_sets['model2'],
all_predictions['model2'])
    sns.heatmap(cm2, annot=True, fmt='d', cmap='Oranges', ax=ax)
    ax.set_title(f"CM: {results[1]['model']}")
    ax.set_ylabel('Истинный класс')
    ax.set_xlabel('Предсказанный класс')

# Confusion Matrix - Модель 3
ax = axes2[2]
if 'model3' in all_predictions:
    cm3 = confusion_matrix(all_test_sets['model3'],
all_predictions['model3'])
    sns.heatmap(cm3, annot=True, fmt='d', cmap='Purples', ax=ax)
    ax.set_title(f"CM: {results[2]['model']}")
    ax.set_ylabel('Истинный класс')
    ax.set_xlabel('Предсказанный класс')

fig4, axes4 = plt.subplots(1, 1, figsize=(8, 8))

# Топ-10 комбинаций параметров
ax = axes4
top_10_results = grid_results.nlargest(10, 'mean_test_score')
param_labels = [
    f"k={row['param_n_neighbors']}, {row['param_metric'][:4]},

```

```

{row['param_weights'][:3]}"}
    for _, row in top_10_results.iterrows()
]
ax.barh(range(10), top_10_results['mean_test_score'], alpha=0.7,
color='mediumpurple')
ax.set_yticks(range(10))
ax.set_yticklabels(param_labels, fontsize=8)
ax.set_xlabel('CV Score')
ax.set_title('Топ-10 комбинаций параметров')
ax.grid(True, alpha=0.3)
ax.invert_yaxis()

plt.tight_layout()

```